

L'objectif de cette procédure est de permettre à une application Python (par exemple un agent IA) d'accéder à une boîte Gmail réelle afin de lire, envoyer et modifier des emails, sans utiliser de mot de passe. Google impose désormais l'utilisation du protocole OAuth 2.0, qui repose sur un mécanisme d'autorisation sécurisé.

La première étape consiste à créer un projet Google Cloud via la Google Cloud Console, puis à activer l'API Gmail, indispensable pour toute interaction avec les emails. Sans cette API, aucune communication avec Gmail n'est possible.

Une fois l'API activée, il est nécessaire de configurer l'écran de consentement OAuth, aussi appelé Branding. Cette étape permet de définir l'identité de l'application, notamment le nom, l'email de support, ainsi qu'une adresse de contact obligatoire. Le type d'utilisateur doit être défini comme Externe, ce qui permet l'accès à un compte Gmail personnel. Tant que ce branding n'est pas finalisé, Google bloque l'accès aux scopes, ce qui empêche toute configuration avancée.

Après le branding, l'étape suivante consiste à ajouter l'utilisateur test dans la section Audience. L'adresse Gmail utilisée par l'application doit impérativement être déclarée ici, car le projet fonctionne initialement en mode test.

Une fois ces prérequis validés, il devient possible d'ajouter les scopes Gmail dans la section Accès aux données. Les scopes définissent précisément les autorisations demandées à l'utilisateur. Dans ce cas, trois autorisations sensibles sont ajoutées manuellement :

gmail.readonly, gmail.send et gmail.modify. Ces scopes permettent respectivement la lecture, l'envoi et la modification des emails.

Lorsque les scopes sont enregistrés, la configuration OAuth peut être finalisée en créant un client OAuth. Dans la section Clients, il faut créer un ID client OAuth de type Application de bureau, qui est le seul format compatible avec une application Python locale. À l'issue de cette création, Google fournit un fichier d'identifiants au format JSON.

Ce fichier, généralement nommé client_secret_XXXX.json, doit être téléchargé, puis renommé en credentials.json et placé dans le répertoire du projet Python. Il contient le client_id et le client_secret, utilisés pour initialiser la connexion OAuth. Ce fichier est strictement confidentiel et ne doit jamais être partagé publiquement.

Lors de la première exécution du script Python, une page Google s'ouvre automatiquement afin que l'utilisateur autorise l'application à accéder à son compte Gmail. Une fois l'autorisation accordée, un fichier token.json est généré localement. Ce token permet aux exécutions suivantes de fonctionner sans intervention humaine, tout en restant sécurisé.

À ce stade, toute la configuration côté Google est terminée. L'application Python est prête à interagir avec Gmail via OAuth 2.0, de manière conforme, sécurisée et pérenne

Intégration de Gmail dans une application Python via OAuth 2.0

Afin de permettre à une application **Python** d'accéder à une **boîte Gmail réelle**, une authentification via **OAuth 2.0** a été mise en place, conformément aux exigences de Google.

Étape 1 — Création du script Python gmail_auth

La première étape consiste à développer un script Python dédié à l'authentification Gmail, nommé **gmail_auth**. Ce script s'appuie sur les bibliothèques officielles de Google et utilise le fichier **credentials.json**, généré depuis la **Google Cloud Console**, afin d'identifier l'application. Les **scopes Gmail** nécessaires sont définis dans le code afin d'autoriser la lecture, l'envoi et la modification des

emails. Ce script est responsable de l'initialisation du flux **OAuth 2.0** et de la gestion automatique des jetons d'accès.

Étape 2 — Déclaration de l'utilisateur dans la section Audience

L'application OAuth étant configurée en **mode test**, il est obligatoire de déclarer explicitement les comptes Gmail autorisés à l'utiliser. Cette opération est réalisée dans la **Google Cloud Console**, section **Audience**, en ajoutant l'adresse email qui sera utilisée par l'agent comme **utilisateur test**. Google autorise jusqu'à **100 utilisateurs** dans ce mode. En l'absence de cette déclaration, toute tentative de connexion entraîne une **erreur 403 (access_denied)**.

Étape 3 — Autorisation via le navigateur web

Lors du **premier lancement** du script **gmail_auth**, l'application détecte l'absence d'un jeton d'accès valide et déclenche automatiquement l'ouverture d'une **page d'autorisation Google** dans le navigateur. L'utilisateur doit alors se connecter à son compte Gmail et accepter les **autorisations demandées** par l'application. Cette validation manuelle est indispensable et n'est requise qu'une seule fois.

Étape 4 — Génération automatique du fichier token.json

Une fois les autorisations acceptées, Google transmet à l'application un **jeton OAuth** valide. Ce jeton est automatiquement enregistré dans un fichier nommé **token.json**. Ce fichier contient à la fois le **token d'accès Gmail** et le **refresh token**, permettant à l'application de conserver un accès permanent sans nécessiter de nouvelle intervention de l'utilisateur. À partir de cette étape, l'authentification est totalement transparente.

Étape 5 — Validation de la connexion Gmail

Afin de confirmer le bon fonctionnement de l'authentification, un appel de test à l'**API Gmail** est exécuté depuis le script Python. La récupération d'un **identifiant de message** et d'une estimation du **nombre total d'emails** présents dans la boîte de réception confirme que la connexion OAuth est pleinement opérationnelle. L'application est alors prête à interagir avec Gmail de manière automatisée.

Si toutes les étapes sont respectées vous verrez ce message dans la console

Autorise l'accès Gmail dans ton navigateur

```
{'messages': [{'id': '19c28a472092d8e9', 'threadId': '19c28a472092d8e9'}], 'nextPageToken': '04770340293294927064', 'resultSizeEstimate': 201}
```